

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP 7.0

PROJECT REPORT ON TAXPAL

TAXPAL: PERSONAL FINANCE & TAX ESTIMATOR

A Smart Full-Stack Solution for Financial Management and Tax Planning

SUBMITTED BY:

Name	Role	Team
Sachin Shekhar	Team Lead / Frontend Developer	Frontend Team
Debjani Roy	Frontend Developer	Frontend Team
Jalal Shaik Jasmine	Frontend Developer	Frontend Team
Varshitha Gopalam	Backend Developer	Backend Team
Rehan Shaik	Backend & API Developer	Backend Team
Jainee Jain	Database Administrator / Modeling	Database Team

ANGULAR FULLSTACK INTERNSHIP PROGRAM

DEVELOPMENT TEAM : BATCH 1 (TEAM 2)

PROJECT DURATION: June 2026 – August 2026

TABLE OF CONTENTS (INDEX)

1. Introduction

2. Project Overview

3. Objectives & Scope

4. Requirements & Technologies Used

- Frontend Tech Stack
- Backend Tech Stack
- Database & Storage
- AI & Email Integrations

5. System Architecture

- Architectural Block Diagram
- Security & Isolation Principles

6. Database Design

- Database Models Schemas
 - User Schema
 - Transaction Schema
 - Budget Schema
 - Category Schema
 - Tax Estimate Schema
 - Alert Schema
 - Chat History Schema
 - Report Schema

7. Module Implementation

- Income & Expense Management
- Categorization & Budgeting
- Tax Estimation Engine
- Reporting & Export

8. Milestone-wise Implementation

- Milestone 1 – Transaction Logging
- Milestone 2 – Categorization & Budgeting
- Milestone 3 – Tax Estimation
- Milestone 4 – Reporting & Export

9. User Interface & Screenshots (Description)

10. API Documentation

- Authentication Route APIs ([/api/auth](#))
- Transaction Route APIs ([/api/transactions](#))
- Budget Route APIs ([/api/budgets](#))
- Category Route APIs ([/api/categories](#))
- Tax Estimate Route APIs ([/api/tax-estimates](#))
- Tax Calendar & Reminders Route APIs ([/api/tax-calendar](#))
- Report Route APIs ([/api/reports](#))
- AI Chatbot Route APIs ([/api/chat](#))
- Dashboard Route APIs ([/api/dashboard](#))

10. Testing & Validation

11. Deployment

12. Challenges & Solutions

13. Team Members & Contributions

14. Future Enhancements

15. Conclusion

16. References

1. INTRODUCTION

With the rapid growth of the gig economy, freelancers, consultants, and self-employed individuals are becoming an important part of today's workforce. However, unlike salaried employees, they do not have taxes automatically deducted from their income. They are responsible for managing multiple income sources, tracking business expenses, maintaining budgets, estimating quarterly taxes, and completing tax-related formalities on their own. Managing these tasks manually can be complicated and may lead to financial mismanagement, missed deadlines, incorrect tax calculations, and unnecessary penalties.

To address these challenges, **TaxPal** was developed as a secure and user-friendly full-stack personal finance platform designed especially for freelancers and self-employed individuals. It provides a centralized system for managing different aspects of personal and professional finances. Users can record and categorize their income and expenses, monitor their spending patterns, and manage monthly budgets effectively.

A major feature of TaxPal is its **quarterly tax estimation system**, which calculates estimated national and state taxes based on factors such as country, location, filing status, income, deductions, and applicable tax slab structures. This helps users understand their potential tax liability and plan their finances accordingly.

The platform also provides **automated reminders** for important financial activities and tax deadlines, reducing the chances of missing critical payments.

In addition, TaxPal allows users to **generate and export financial reports**, making it easier to maintain organized financial records. An integrated **AI assistant** further enhances the platform by providing contextual financial guidance and helping users better understand their financial situation.

Overall, TaxPal brings **income and expense tracking, budgeting, tax estimation, reminders, reporting, and AI-based assistance** together in a single platform. Its goal is to simplify financial management, reduce manual effort, and help independent workers make better and more informed financial decisions.

2. PROJECT OVERVIEW

TaxPal is structured as an integrated web application consisting of a modular Angular frontend, a robust Express.js/Node.js backend, and a cloud-hosted MongoDB Atlas database.

Core Features

- **Comprehensive Ledger:** Log and categorize income and expenses with detailed transaction information, making it easier to maintain an organized record of financial activities.
- **Budget Performance Analyzer:** Configure monthly spending limits for different categories and monitor budget utilization through progress meters and variance indicators.
- **Tax Calculation Engine:** Compute annualized and quarterly tax obligations by considering income, applicable tax slabs, and deductions such as business expenses, retirement contributions, health insurance, and home office costs.
- **Automated Alerts & Calendars:** Receive automated notifications 7 days and 1 day before important tax due dates, along with detailed calendar entries to help users stay on schedule.
- **Interactive Reports:** Generate professionally structured PDF reports using **PDFKit** and customized CSV files, allowing users to review, download, and maintain their financial records conveniently.
- **Gemini-Powered Financial Assistant:** Engage in natural-language conversations to analyze budgets, understand financial patterns, receive tax-related guidance, generate reports, and perform supported financial operations through an AI-assisted interface.
- **Secure Full-Stack Architecture:** The application follows a modular architecture with an **Angular frontend, Express.js/Node.js backend, and MongoDB Atlas database**, ensuring organized data management, scalability, and secure communication between different layers of the system.

3. OBJECTIVES & SCOPE

Objective

1. **Financial Literacy & Clarity:** Provide self-employed individuals with real-time feedback on their income, expenses, and net profit margins, helping them understand their overall financial position.
2. **Automated Tax Management:** Simplify tax estimation by reducing complex calculations to a few parameter selections, taking the guesswork out of quarterly tax planning.
3. **Budget Compliance:** Actively monitor category-wise spending and warn users when their expenditures approach or cross predefined budget limits.
4. **AI-Enabled Automation:** Provide an intelligent chat interface that can analyze financial information and execute supported app commands, such as generating reports, emailing financial reports, and downloading files.
5. **Centralized Financial Management:** Bring income tracking, expense management, budgeting, tax estimation, reminders, and reporting together within a single platform.
6. **Improved Financial Decision-Making:** Provide users with meaningful insights and organized financial reports to help them identify spending patterns, control unnecessary expenses, and plan their finances more effectively

Scope

- **Supported Countries for Tax Slabs:** The platform currently supports tax calculations for the **United States, India, Canada, Australia, and the United Kingdom**, with applicable regional tax rules where available.
- **Secure Environment:** API keys, database credentials, passwords, and other sensitive configuration values are handled strictly on the server side using secure environment variables.
- **User-Specific Data Access:** Database operations are scoped to authenticated users, ensuring that each user can only view, modify, and interact with their own financial records.
- **Automated Email Alerts:** Tax reminders and important financial notifications can be delivered automatically using the **Resend API and SMTP** services.
- **Financial Reporting & Export:** Users can generate structured financial reports and export their records in formats such as **PDF and CSV** for future reference, documentation, or tax preparation.

4. REQUIREMENTS & TECHNOLOGIES USED

TaxPal leverages modern, light-weight, and highly integrable technologies across the web application stack:

Frontend Tech Stack

- **Angular 18+**: Framework for structured components, models, routing, and HTTP communication.
- **TypeScript**: Typed language layer for solid client logic and data interface definitions.
- **HTML5 & CSS3**: Vanilla styles containing custom-tailored dark and light variables, eliminating bloated frameworks.

Backend Tech Stack

- **Node.js**: Asynchronous, event-driven runtime environment.
- **Express.js**: Minimalist web application router to design RESTful APIs.
- **JSON Web Tokens (JWT)**: Secure user session state transfer in an stateless architecture.

Database & Storage

- **MongoDB Atlas**: Fully-managed cloud NoSQL database storing structured JSON-like documents.
- **Mongoose ORM**: Object data modeling to define schemas, cast types, build hooks, and run validations.

AI & Email Integrations

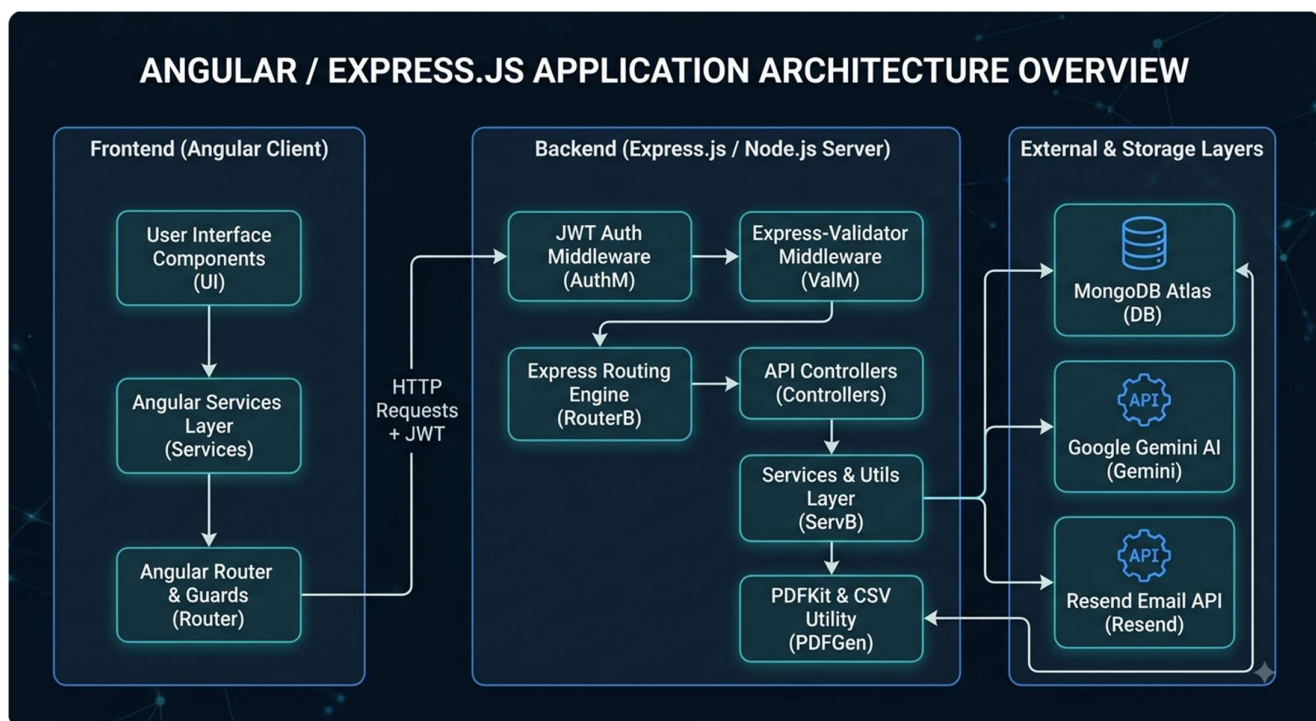
- **Google Gemini API (`gemini-2.5-flash`)**: Advanced large language model handling conversational queries and structural commands.
- **Resend API / SMTP**: Automated transactional mailing for reminders, registration confirmations, and report deliveries.

5. SYSTEM ARCHITECTURE

The TaxPal application follows a clean, decoupled **Client-Server Architecture** utilizing REST APIs for message passing. The frontend communicates with the backend via HTTP and WebSockets (if applicable).

The server handles routing, request validation, authentication, and execution of business logic, serving as the intermediate layer between database nodes and public cloud APIs.

Architectural Block Diagram



Security & Isolation Principles

- No Client-Side API Leakage:** Gemini and Resend keys are strictly confined to the backend environment variables (`.env`).
- User Data Isolation:** Every controller queries MongoDB with `userId: req.user.id`, preventing unauthorized access.

Stateless Authentication: Passwords are encrypted using high-entropy hashing algorithms, and sessions are verified using cryptographic signatures in request headers.

6. DATABASE DESIGN

Database Models Schemas

The database structure relies on standard, indexed schemas in MongoDB configured with Mongoose.

User Schema

- **_id**: ObjectId (Primary Key)
- **name**: String (Required)
- **username**: String (Unique, Required)
- **email**: String (Unique, Required)
- **password**: String (Encrypted, Required)
- **country**: String (Default: "")
- **income_bracket**: String (Enum: ["low", "middle", "high", ""])
- **notificationPreferences**: Object containing toggles for emailAlerts, weeklyDigest, budgetAlerts, taxDeadlines, and taxPaymentConfirmation.

Transaction Schema

- **_id**: ObjectId (Primary Key)
- **userId**: ObjectId (Foreign Key referencing User, Indexed)
- **type**: String (Enum: ["income", "expense"], Required)
- **category**: String (Required)
- **amount**: Number (Required)
- **date**: Date (Required)
- **description**: String
- **notes**: String
- **timestamps**: createdAt, updatedAt

Budget Schema

- **_id**: ObjectId (Primary Key)
- **userId**: ObjectId (Foreign Key referencing User, Indexed)
- **category**: String (Required)

- **limit:** Number (Required, Minimum 1)
- **month:** String (Required, format: "YYYY-MM")
- **Indexes:** Compound unique index on { `userId: 1`, `category: 1`, `month: 1` } to prevent multiple budgets for the same category in a month.

Category Schema

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (Foreign Key referencing User, Indexed)
- **name:** String (Required)
- **type:** String (Enum: ["income", "expense"], Required)
- **isDefault:** Boolean (Default: false)
- **color:** String (Hex color for charts)
- **Indexes:** Compound unique index on { `userId: 1`, `name: 1`, `type: 1` } to prevent duplicates.

Tax Estimate Schema

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (Foreign Key referencing User, Indexed)
- **country:** String (Default: "IN")
- **state:** String
- **filingStatus:** String (Default: "Single")
- **quarter:** String (Enum: ["Q1", "Q2", "Q3", "Q4"], Required)
- **year:** Number (Default: Current Year)
- **grossIncome:** Number
- **businessExpenses:** Number
- **retirementContribution:** Number
- **healthInsurancePremiums:** Number
- **homeOfficeDeduction:** Number
- **totalDeductions:** Number
- **taxableIncome:** Number
- **nationalTax:** Number
- **stateTax:** Number
- **estimatedTax:** Number

- **effectiveTaxRate:** Number
- **dueDate:** Date
- **paymentStatus:** String (Enum: ["Pending", "Completed"], Default: "Pending")
- **paymentCompletedAt:** Date
- **Indexes:** Compound index on { `userId: 1, quarter: 1, year: 1` }.

Alert Schema

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (Foreign Key referencing User, Indexed)
- **type:** String (Enum: ["reminder", "payment"], Required)
- **message:** String (Required)
- **alertDate:** Date (Required)
- **isRead:** Boolean (Default: false)
-

Chat History Schema

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (Foreign Key referencing User, Indexed)
- **role:** String (Enum: ["user", "model"])
- **message:** String
- **actions:** Array (Navigation/Download structures)
- **timestamp:** Date
-

Report Schema

- **_id:** ObjectId (Primary Key)
- **userId:** ObjectId (Foreign Key referencing User, Indexed)
- **reportName:** String (Required)
- **reportType:** String (Enum of configurations e.g., `income_statement`, `tax_summary`, `budget_performance`, etc.)
- **period:** String (Required)
- **format:** String (Enum: ["PDF", "CSV"], Required)
- **filePath:** String (Relative path to storage location on server disk)
- **data:** Mixed (Raw data dump representing variables printed in report tables)

taxestimates	
tax_id	objectId
tax	int
businessExpens.	int
confirmations.	bool
confirmations.	date
country	string
createdAt	date
deductions	int
deductions	date
effectiveTax.	(float)
estimatedTax	(float)
filingStatus	string
grossIncome	int
grossIncomePer.	int
healthInsurance.	int
homeOfficeDedu.	int
interest	bool
nationalTax	(float)
paymentComple.	(float)
paymentStatus	string
quarter	string
reminderEmails.	bool
reminderEmails.	date
retirementCont.	int
state	string
year	int
taxableIncome	int
totalDeductions	int
updatedAt	date
userId	objectId
year	int

budgets	
tax_id	objectId
tax	int
category	string
createdAt	date
limit	int
month	string
updatedAt	date
userId	objectId

categories	
tax_id	objectId
tax	int
color	string
createdAt	date
isDefault	bool
name	string
type	string
updatedAt	date
userId	objectId

users	
tax_id	objectId
tax	int
country	string
createdAt	date
email	string
income_bracket	string
name	string
notification.	[]
budgetAlerts	bool
emailAlerts	bool
healthInsur.	bool
taxDeductions	bool
taxPaymentCon.	bool
weeklyDigest	bool
password	string
updatedAt	date
username	string

transactions	
tax_id	objectId
tax	int
amount	int
category	string
createdAt	date
date	date
description	string
notes	string
type	string
updatedAt	date
userId	objectId

reports	
tax_id	objectId
tax	int
createdAt	date
data	()
country	string
currencySyml	string
deductionsRe.	()
businessExp.	int
healthInsur.	int
homeOffice	int
retirement	int
expensesBreakd.	()
amount	int
category	string
total	int
header	()
currencySym.	string
generatedAt.	string
logo	string
periodLabel	string
title	string
userEmail	string
username	string
incomeBreakd.	()
amount	int
category	string
total	int
isActiveAtina.	bool
metrics	()
estimatedTax	(float)
grossIncome	int
netIncome	int
taxableInco.	int
totalDeduct.	int
totalExpens.	int
totalIncome	int
taxCalculation.	()
date	string
effectiveTa.	double
nationalTax	(float)
stateTax	int
filePath	string
format	string
period	string
reportName	string
reportType	string
updatedAt	date
userId	objectId

chathistories	
tax_id	objectId
tax	int
action	(float)
data	()
totalIncome	int
totalExpens.	int
totalIncome	int
transaction.	int
domainAddr1	string
emailRequested	bool
emailStatus	()
message	string
sent	bool
filePath	string
filePath	string
periodLabel	string
reportId	objectId
reportName	string
summary	()
currencySym.	string
expenses	int
income	int
savings	int
type	string
view	string
createdAt	date
message	string
role	string
updatedAt	date
userId	objectId

7. MODULE IMPLEMENTATION

Income & Expense Management

This module registers all financial inputs. Users can categorize, add metadata, and track transactions chronologically.

- **Backend Processing:** The `transactionController` processes incoming additions, runs validations (e.g. valid amounts and dates), and links them to the active JWT user token.
- **Frontend Rendering:** Displayed in dynamic transaction tables with features to search, filter by category/type, and delete or update ledger rows.

Categorization & Budgeting

Users can customize spending tags and establish monthly financial parameters.

- **Categories:** Divided into default categories (e.g., *Salary, Consulting, Food, Travel, Rent, Utilities*) and custom categories created by users.
- **Budget Metrics:** Calculates budget performance by querying expenses logged in a category against the monthly limit.
$$\text{Remaining Budget} = \text{Budget Limit} - \text{Total Expenses in Month}$$
 Variance is styled dynamically: green for within limits and red for over-budget states.

Tax Estimation Engine

Calculates projected income tax based on geographical rules and deductions:

- **Deduction Aggregation:** Deducts allowable expenses from gross income:
$$\text{Total Deductions} = \text{Business Expenses} + \text{Retirement} + \text{Health Insurance} + \text{Home Office}$$

$$\text{Taxable Income} = \max(0, \text{Gross Income} - \text{Total Deductions})$$
- **Annualization Technique:** To estimate the tax bracket for a quarter, the engine annualizes the income:
$$\text{Annualized Taxable Income} = \text{Quarterly Taxable Income} \times 4$$
 The annualized income is passed through slab structures for the selected country (United States, India, Canada, Australia, United Kingdom) and filing status (Single, Married Joint, etc.) to get the annualized tax.
- **Quarterly Distribution:** The final quarterly estimated tax is:
$$\text{Quarterly Estimated Tax} = \frac{\text{Annualized Tax}}{4} + \text{State Tax}$$
 (State taxes are calculated as a flat rate based on state tax listings)

Reporting & Export

Enables exporting financial data into files for offline accounting:

- **CSV Exports:** Handled using a custom string builder to structure ledger lists into clean comma-separated formats, supporting instant downloads.
- **PDF Reports:** Uses `pdfkit` on the backend to dynamically generate documents. Each document is styled with headings, colored metric cards, and structured tables.

8. MILESTONE-WISE IMPLEMENTATION

Milestone 1 – Transaction Logging

- **Timeline:** 30 June 2026 – 9 July 2026
- **Key Tasks:**
 - Environment setup (Node.js backend, Angular frontend boilerplate, MongoDB Atlas configuration).
 - Database schema definitions (User and Transaction collections).
 - JWT Authentication implementation (login, signup, auth guards).
 - Transaction ledger APIs (CRUD operations for income and expenses).
 - E2E testing and fixing field mismatches (frontend-backend alignment).

Milestone 2 – Categorization & Budgeting

- **Timeline:** 10 July 2026 – 23 July 2026
- **Key Tasks:**
 - Category database collection configurations.
 - Default categories generation on registration.
 - Category-based budget setup (Limit, Category, Month configuration).
 - Budget Progress computations and visualization on dashboards.
 - Validation rules configuration to prevent budget duplicate entries.

Milestone 3 – Tax Estimation

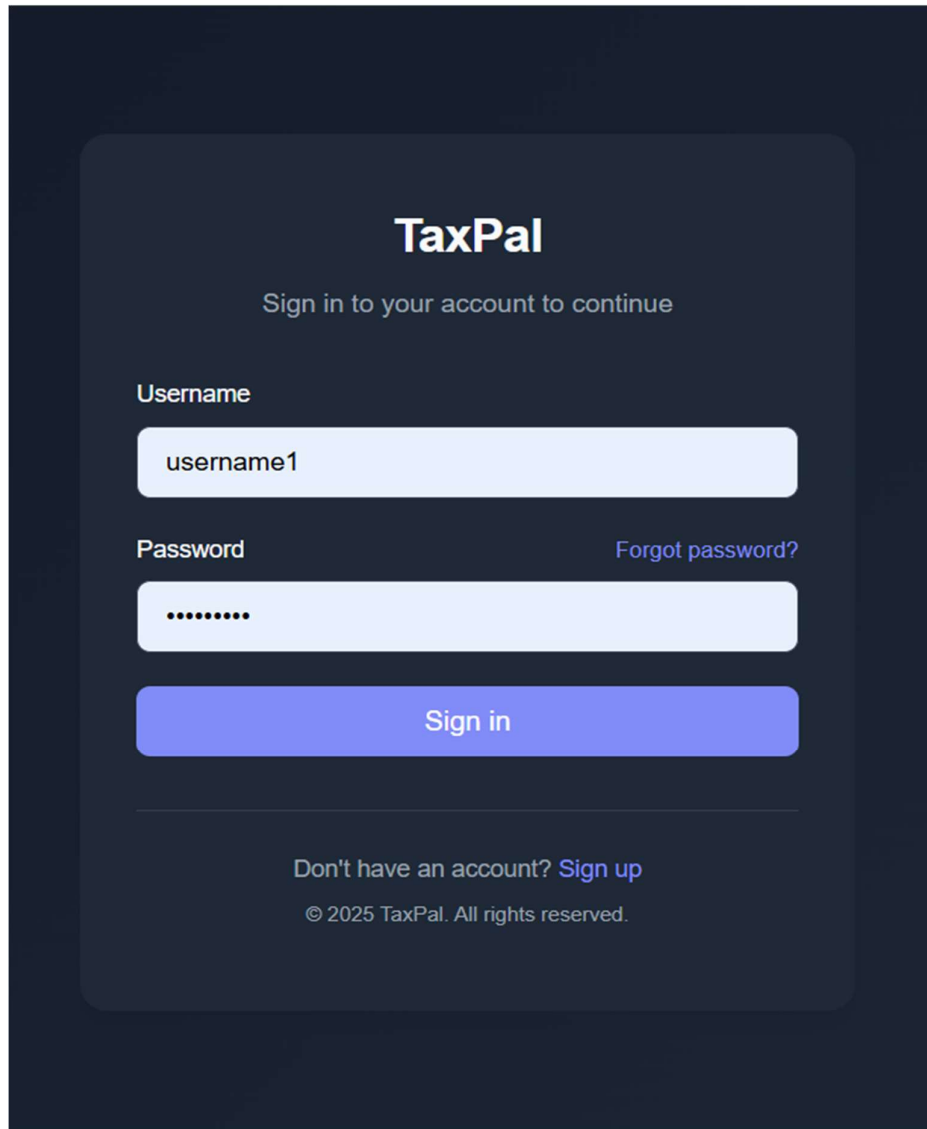
- **Timeline:** 24 July 2026 – 5 August 2026
- **Key Tasks:**
 - Tax calculation utilities (slab definitions for India, US, Canada, Australia, UK).
 - Deductions and Taxable Income calculations.
 - Quarterly Tax Estimates and Tax Calendars.
 - Upcoming payment notifications (7-day and 1-day reminders via Resend API).
 - Interactive payment completion dashboard hooks.

Milestone 4 – Reporting & Export

- **Timeline:** 6 August 2026 – 17 August 2026
- **Key Tasks:**
 - PDF Generation backend service with `pdfkit` drawing grids and tables.
 - Custom CSV Spreadsheet exporter.
 - Google Gemini API integration (`gemini-2.5-flash`) for AI Financial Chatbot.
 - Natural Language queries mapping to user financial documents.
 - Email forwarding capabilities attaching generated reports via SMTP.

9. USER INTERFACE & SCREENSHOTS

1. **Login & Registration Pages:** Sleek, form-based layout with real-time field validation, password strength meters, and region/income bracket selectors.



A sleek, dark-themed login page for TaxPal. The page features a central light blue rounded rectangle containing the login form. At the top, the 'TaxPal' logo is displayed in white. Below it, the text 'Sign in to your account to continue' is shown in a smaller white font. The form includes a 'Username' label and a text input field containing 'username1'. Below this is a 'Password' label and a text input field filled with dots. To the right of the password field is a 'Forgot password?' link in blue. A large blue 'Sign in' button is positioned below the password field. At the bottom of the form, there is a horizontal line followed by the text 'Don't have an account? Sign up' where 'Sign up' is a blue link. The footer of the form contains the copyright notice '© 2025 TaxPal. All rights reserved.' in a small white font.

TaxPal

Sign in to your account to continue

Username

username1

Password

Forgot password?

.....

Sign in

Don't have an account? [Sign up](#)

© 2025 TaxPal. All rights reserved.

TaxPal

Create an Account

Enter your information to create your TaxPal account

Username

username

Password

Full Name

Enter your full name

Email

Enter your email address

Country

Select your country



Income Bracket (Optional)

Select your income bracket



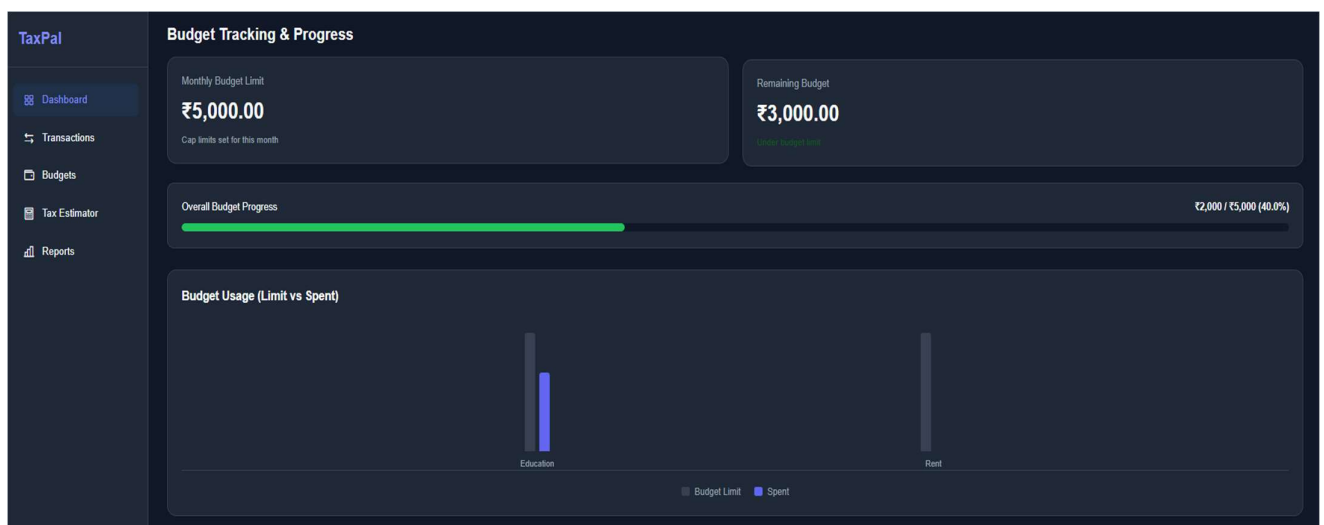
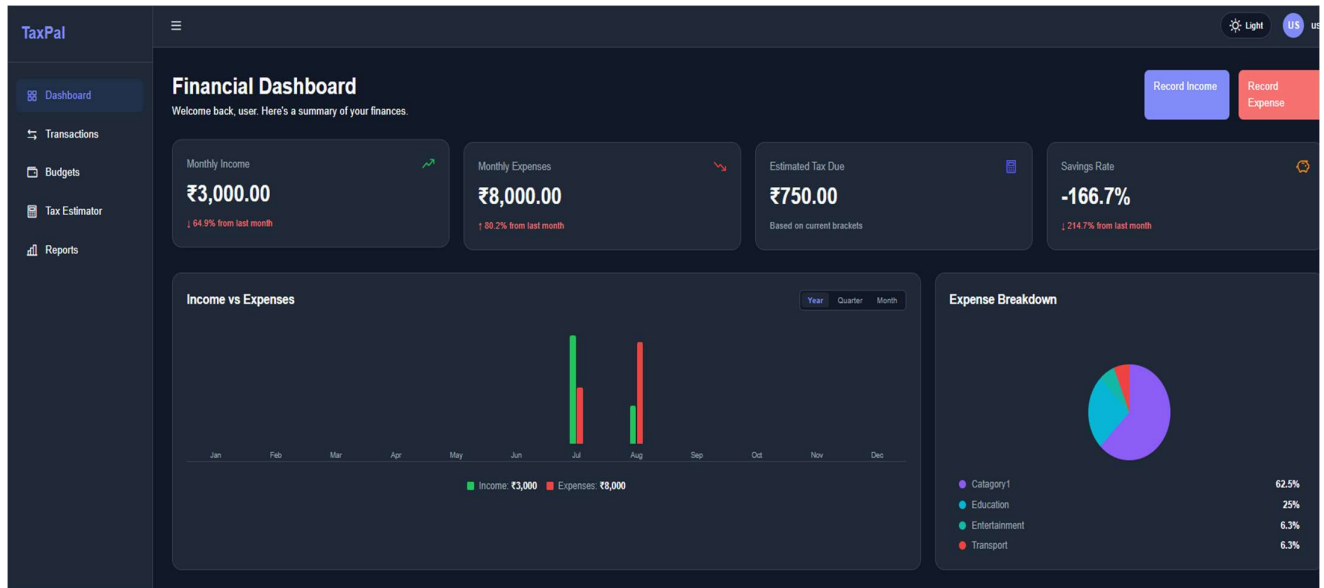
Create Account

Already have an account? [Sign in](#)

By creating an account, you agree to our [Terms of Service](#) and [Privacy Policy](#).

2. Main Dashboard: A dashboard containing:

- Dynamic cards showing *Total Income*, *Total Expenses*, and *Net Balance*.
- A categorical distribution chart (pie/doughnut style) visualizing spending.
- Interactive lists showing recent transactions.



3. **Transaction Manager:** Table configuration with filter dropdowns for Category, Transaction Type, and date range query fields, complemented by modal forms to Add/Edit/Delete transactions.

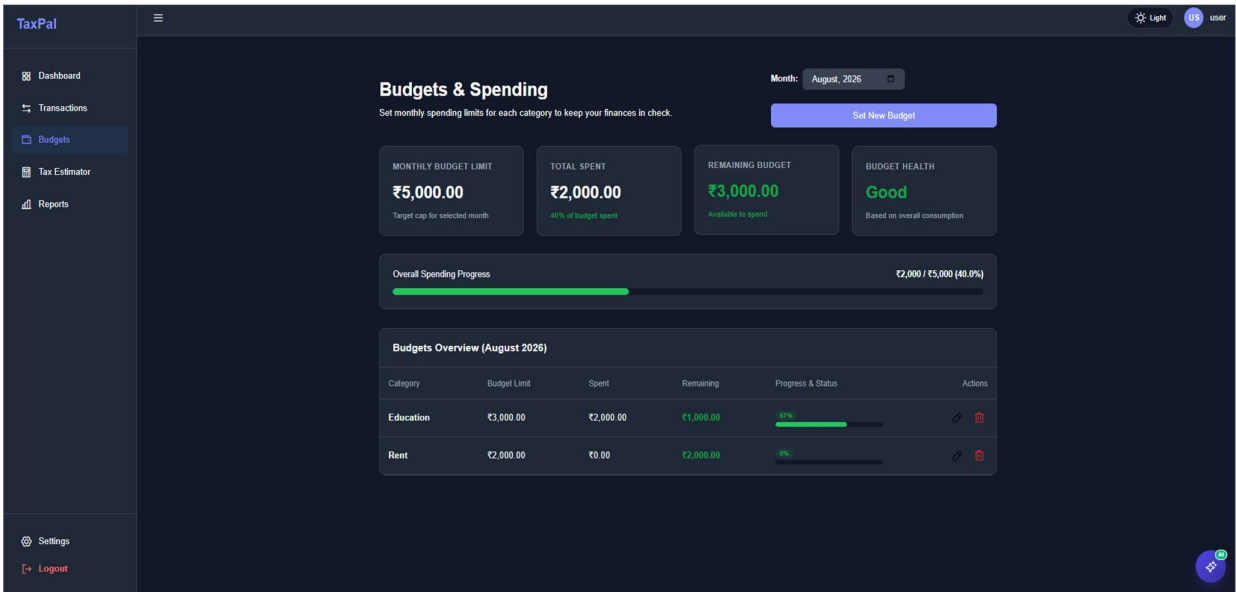
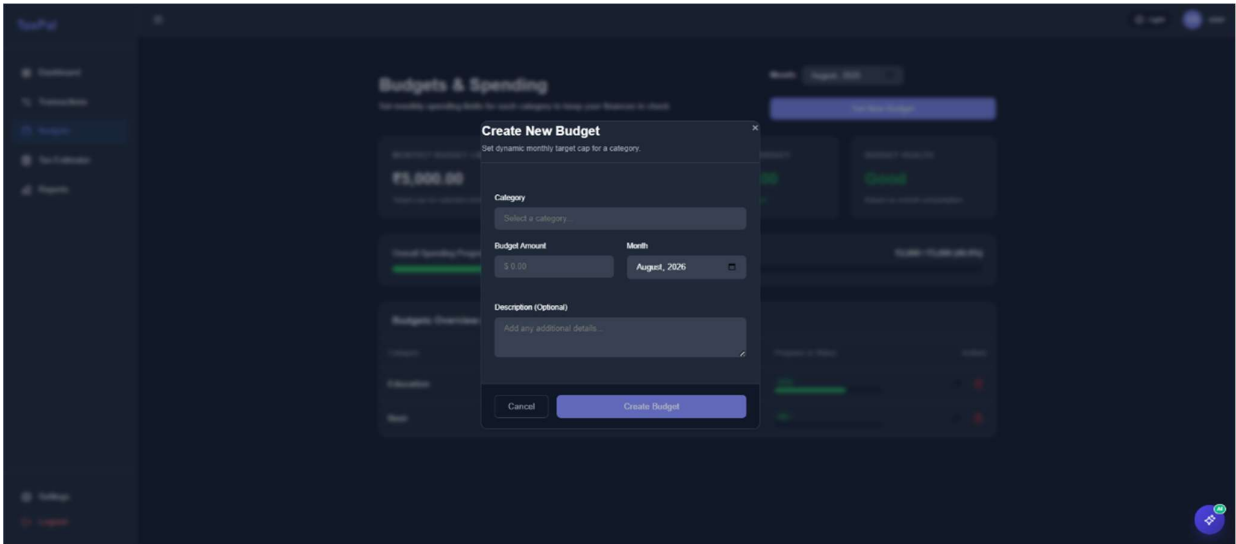
The screenshot shows the 'Record New Income' modal form. It has a title bar with a close button. Below the title is a subtitle: 'Add details about your income to track your finances better.' The form contains several input fields: 'Description' (with a placeholder 'e.g. Web Design Project'), 'Amount' (with a placeholder '\$ 0.00'), 'Category' (with a placeholder 'Search or type category'), and 'Date' (with a placeholder 'dd-mm-yyyy' and a calendar icon). There is also a 'Notes (Optional)' section with a text area and a placeholder 'Add any additional details.' At the bottom are 'Cancel' and 'Save Income' buttons.

The screenshot shows the 'Record New Expense' modal form. It has a title bar with a close button. Below the title is a subtitle: 'Add details about your expense to track your finances better.' The form contains several input fields: 'Description' (with a placeholder 'e.g. Web Design Project'), 'Amount' (with a placeholder '\$ 0.00'), 'Category' (with a placeholder 'Search or type category'), and 'Date' (with a placeholder 'dd-mm-yyyy' and a calendar icon). There is also a 'Notes (Optional)' section with a text area and a placeholder 'Add any additional details.' At the bottom are 'Cancel' and 'Save Expense' buttons.

The screenshot shows the 'All Transactions' table. It has a title bar with a close button. Below the title is a subtitle: 'View and manage your recent income and expenses.' The table has two tabs: 'Income' (selected) and 'Expense'. The table has columns: 'Date', 'Description', 'Category', 'Type', 'Amount', and 'Actions'. The 'Type' column has a dropdown menu with 'Income' and 'Expense' options. The 'Amount' column shows values with signs (+/-). The 'Actions' column has 'Edit' and 'Delete' buttons.

Date	Description	Category	Type	Amount	Actions
Aug 6, 2026	Edu	Education	Income	+2,000.00	Edit Delete
Aug 5, 2026	client	Freelancing	Income	+1,000.00	Edit Delete
Aug 4, 2026	Movie	Entertainment	Expense	-500.00	Edit Delete
Aug 4, 2026	Bus	Transport	Expense	-500.00	Edit Delete
Aug 3, 2026	business	Business	Income	+2,000.00	Edit Delete
Aug 2, 2026	cat1	Category1	Expense	-5,000.00	Edit Delete
Jul 22, 2026	Freelancing	Freelancing	Income	+8,542.00	Edit Delete
Jul 14, 2026	Education	Education	Income	+1,440.00	Edit Delete

4. **Budget Planner:** Visual progress meters detailing budget performance per category, highlighting Limit exceeded



5. **Tax Estimator:** Interactive form where self-employed users input deductions to view a detailed calculation breakdown (National Tax, State Tax, Tax Bracket, and Target Payment Date).

The screenshot shows the 'TaxPal' application interface. On the left is a dark sidebar with navigation links: Dashboard, Transactions, Budgets, Tax Estimator (highlighted), Reports, Settings, and Logout. The main content area is titled 'Tax Estimator' and includes a sub-header 'Calculate your estimated liability for obligations based on income, deductions, and local brackets.' Below this is a 'Quick Estimate Tax Calculator' form with fields for 'Country/Region' (set to 'Canada'), 'Filing Status' (set to 'Single'), and 'Tax Year' (set to 'Q2 (July-Sep 2020)'). There are also checkboxes for 'Include Offsets' and 'Include Payments', and a 'Calculate Estimated Tax' button. To the right of the form is a 'Tax Summary' box showing a total liability of '\$29,847.31' and a 'Tax Bracket' of '24.00%'. Below the summary is a 'Recent History' table with two entries: 'Q2 2020 - Canada (2020)' and 'Q2 2020 - Canada (2020)'. At the bottom of the main area is a 'Tax Calendar' section showing a calendar for the year 2020 with dates for 'Q2 2020' and 'Q3 2020'.

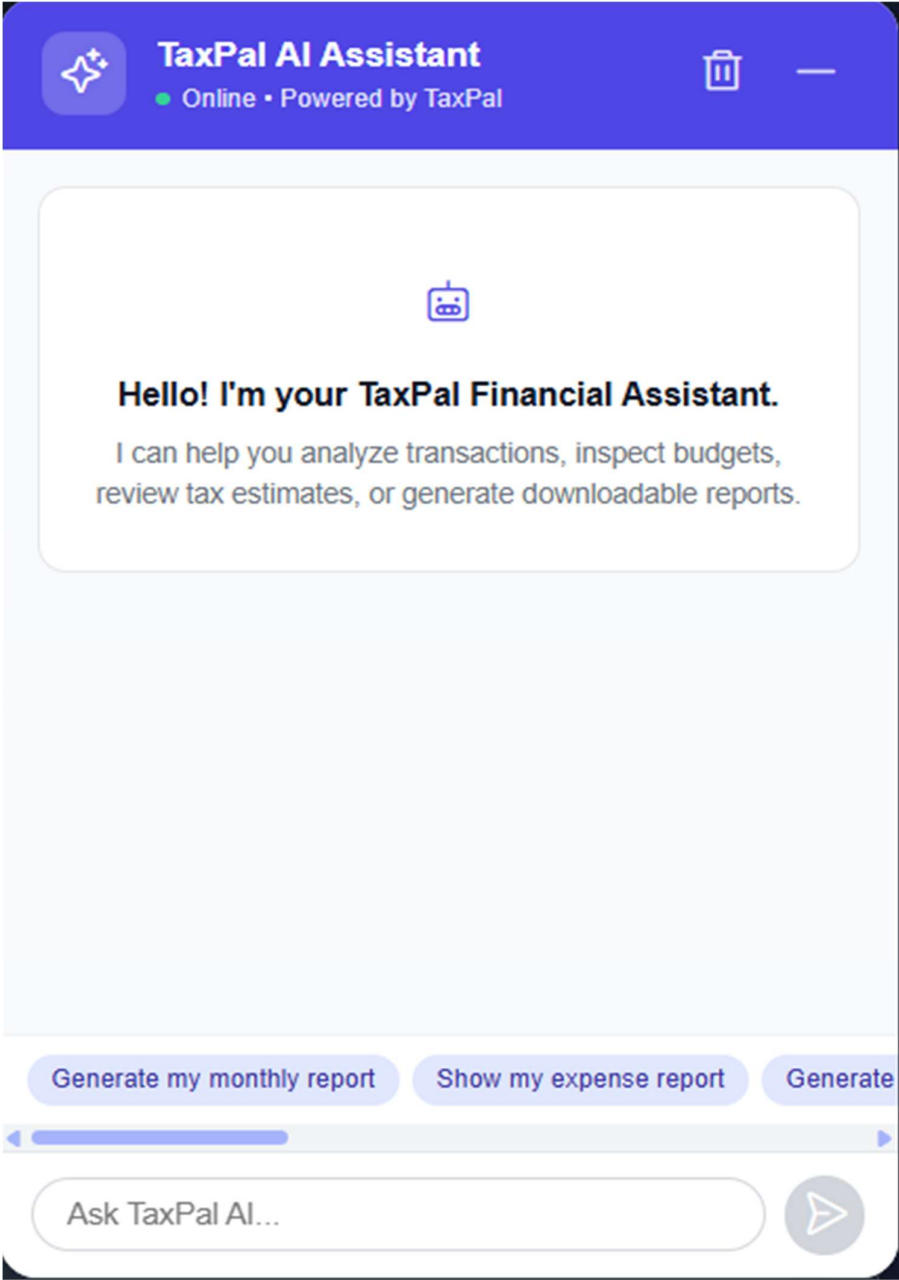
6. **Reports & Exports:** File catalog with option keys to compile reports as PDF/CSV, download files locally, or email them directly to their registered email

The screenshot shows the 'TaxPal' application interface. On the left is a dark sidebar with navigation links: Dashboard, Transactions, Budgets, Tax Estimator, Reports (highlighted), Settings, and Logout. The main content area is titled 'Financial Reports' and includes a sub-header 'Generate and download your financial reports.' Below this is a 'Generate Report' form with fields for 'Report Type' (set to 'Income Statement'), 'Period' (set to 'Current Month'), and 'Format' (set to 'PDF'). There are 'Reset' and 'Generate Report' buttons. Below the form is a 'Recent Reports' table with the following data:

Report Name	Generated	Period	Format	Actions
Income Report - August 2020	Aug 18, 2020, 07:42 PM	August 2020	PDF	View, Download, Email
Income Statement - August 2020	Aug 17, 2020, 05:49 PM	August 2020	CSV	View, Download, Email
Tax Summary - Q2 2020	Aug 16, 2020, 04:57 PM	Q2 2020	PDF	View, Download, Email
Income Statement - August 2020	Aug 16, 2020, 03:57 PM	August 2020	CSV	View, Download, Email

Below the table is a large box with the text 'Select a report to preview' and 'Generated reports will appear here for review before downloading.'

7. **AI Chatbot Bubble:** Floating drawer on the bottom-right corner housing.



10. API DOCUMENTATION

All endpoints are hosted at a local or production server URI. Secured routes require an `Authorization` header containing the JWT token:

```
Authorization: Bearer <your_jwt_token>
```

Authentication Route APIs (`/api/auth`)

- `POST /api/auth/register` - Create user account (inputs: name, username, email, password, country, incomeBracket).
- `POST /api/auth/login` - Authenticates credentials and returns a JWT token.
- `GET /api/auth/me` - Returns logged-in user profile details (protected).
- `PUT /api/auth/notifications` - Modifies notification toggles (protected).

Transaction Route APIs (`/api/transactions`)

- `POST /api/transactions` - Log a transaction (protected; inputs: type, category, amount, date, description, notes).
- `GET /api/transactions` - Fetch user transaction history with search & filter parameters (protected).
- `GET /api/transactions/:id` - Fetch transaction details (protected).
- `PUT /api/transactions/:id` - Update transaction properties (protected).
- `DELETE /api/transactions/:id` - Remove transaction (protected).

Budget Route APIs (`/api/budgets`)

- `GET /api/budgets/progress` - Fetch spending metrics vs budget limits (protected).
- `POST /api/budgets` - Set monthly category budget limits (protected).
- `GET /api/budgets` - Get all budgets (protected).

- GET /api/budgets/:id - Get budget details (protected).
- PUT /api/budgets/:id - Update budget limits (protected).
- DELETE /api/budgets/:id - Delete budget (protected).

Category Route APIs (/api/categories)

- GET /api/categories/default - Retrieve global fallback system categories.
- POST /api/categories/suggest - Receive AI suggestions based on transaction description.
- POST /api/categories - Create custom category (protected).
- GET /api/categories - Fetch custom and default categories (protected).
- GET /api/categories/:id - Fetch details of a category (protected).
- PUT /api/categories/:id - Update category (protected).
- DELETE /api/categories/:id - Delete category (protected).

Tax Estimate Route APIs (/api/tax-estimates)

- POST /api/tax-estimates - Calculate and save tax calculation (protected).
- GET /api/tax-estimates - Retrieve past tax calculations (protected).
- GET /api/tax-estimates/:id - Retrieve details of a tax estimate (protected).
- PUT /api/tax-estimates/:id - Modify parameters of a tax record (protected).
- DELETE /api/tax-estimates/:id - Delete tax calculation history (protected).

Tax Calendar & Reminders Route APIs (/api/tax-calendar)

- GET /api/tax-calendar - Get all tax events and alerts (protected).
- GET /api/tax-calendar/upcoming - Get upcoming tax payment deadlines (protected).
- PATCH /api/tax-calendar/:id/read - Mark reminder alert as read (protected).
- PATCH /api/tax-calendar/:id/unread - Mark reminder alert as unread (protected).
- PATCH /api/tax-calendar/:id/payment - Complete quarterly payment (protected).

- PATCH /api/tax-calendar/:id/payment/undo - Undo payment completion status (protected).
- POST /api/tax-calendar/trigger-reminders - Manually trigger tax reminder emails (developer webhook).

Report Route APIs (/api/reports)

- GET /api/reports - List metadata of all reports (protected).
- POST /api/reports - Compile new CSV or PDF report (protected).
- DELETE /api/reports/:id - Delete generated report from storage (protected).
- GET /api/reports/:id/download - Fetch report file as download attachment.
- POST /api/reports/email or POST /api/reports/:id/email - Send report to user's registered email (protected).

AI Chatbot Route APIs (/api/chat)

- POST /api/chat - Process natural language query and return scoped response (protected).
- GET /api/chat/history - Fetch conversation log (protected).
- DELETE /api/chat/history - Clear active user conversation log (protected).
- GET /api/chat/suggestions - Fetch suggested query cards based on user data (protected).

Dashboard Route APIs (/api/dashboard)

- GET /api/dashboard/summary - Fetch dashboard balance summary cards (protected).
- GET /api/dashboard/recent - Fetch recent transactions (protected).
- GET /api/dashboard/spending-breakdown - Fetch spending category distribution (protected).

11. TESTING & VALIDATION

To ensure stability, reliability, and consistency across the entire application stack, the team followed a structured testing and validation workflow covering the frontend, backend APIs, database operations, and tax calculation logic.

1. **API Validation Testing:** Developed a comprehensive Postman collection, such as `Authentication.postman_collection.json`, to verify API response status codes, request and response payload structures, validation messages, authentication, and authorization mechanisms. This helped ensure that APIs behaved correctly for both valid and invalid requests.
2. **Integration Testing:** The integration between the Angular frontend and Express.js backend was carefully tested. Sachin Shekhar (Team Lead) audited local builds to identify inconsistencies between Angular service fields and Express controller parameters. Mismatched keys such as `incomeBracket` and `income_bracket` were identified and synchronized to ensure smooth data exchange.
3. **Data Scoping Verification:** Security tests were performed to ensure that users could access only their own financial records. For example, requests were simulated where User B attempted to retrieve or modify transactions belonging to User A. The backend correctly rejected unauthorized requests by returning appropriate responses such as **404 Not Found** or **401 Unauthorized**.
4. **Boundary Value Testing:** The tax calculation module was tested with unusual and extreme inputs, including negative incomes, zero deductions, very high incomes, and extreme tax bracket values. These tests helped verify that the calculator remained stable and handled invalid or unexpected inputs without producing incorrect results.
5. **Frontend Validation Testing:** Forms and user interactions were tested to ensure that required fields, invalid inputs, incorrect formats, and submission errors were handled properly. This helped improve the overall usability and prevented invalid data from being submitted to the backend.
6. **Error Handling & Regression Testing:** Common failure scenarios, including invalid API requests, database errors, expired authentication, and incorrect input values, were tested to ensure meaningful error messages were displayed. Previously implemented features were also re-tested after major changes to prevent new updates from breaking existing functionality.

12. DEPLOYMENT

The TaxPal application was deployed as a full-stack cloud-based application consisting of an Angular frontend, Node.js and Express.js backend, and MongoDB Atlas database. The deployment process was carried out to make the complete application accessible through the cloud and to ensure proper communication between all application components.

- **12.1 Database Deployment**

MongoDB Atlas was configured as the cloud database for the TaxPal application. The required database collections and models were configured and connected to the backend using the MongoDB Atlas connection string. The database connection was tested to ensure that the deployed backend could successfully communicate with the database.

- **12.2 Backend Deployment**

The backend was developed using Node.js and Express.js and deployed using Vercel. The backend was configured to handle API requests and route them to the Express application using the vercel.json configuration file.

The backend deployment configuration was:

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/app.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/api/(.*)",
      "dest": "/src/app.js"
    }
  ]
}
```

After deployment, the backend APIs were tested to verify authentication, database connectivity, transaction management, budget management, tax estimation, reminder functionality, report generation, and AI chatbot integration.

- **12.3 Environment Configuration**

The required environment variables were configured in the deployment environment to securely manage

database credentials, authentication secrets, AI services, and email services.

The following environment variables were configured:

- **MONGO_URI:** MongoDB Atlas connection string used for database connectivity.
- **JWT_SECRET:** Secret key used for JWT-based authentication.
- **GEMINI_API_KEY:** API key used for the TaxPal AI chatbot.
- **RESEND_API_KEY:** API key used for email notifications and report delivery.

Sensitive credentials were not included directly in the source code and were maintained through environment variables.

- **12.4 Frontend Deployment**

The frontend was developed using Angular and prepared for production deployment. Before deployment, the Angular application was compiled using the production build command:

`ng build --configuration production`

The generated production build was then deployed to the hosting platform. The frontend was configured to communicate with the deployed backend through the appropriate API endpoints.

- **12.5 Frontend and Backend Integration**

After deploying both components, the frontend was connected to the deployed backend APIs. End-to-end testing was performed to verify that requests from the Angular frontend were correctly processed by the Express backend and that the backend could retrieve and update data in MongoDB Atlas.

The following major functionalities were verified after deployment:

- User registration and login
- JWT authentication
- Transaction management
- Budget management
- Category management
- Tax estimation
- Tax calendar and reminders
- Financial reports
- AI chatbot
- Email notifications
- Report generation and download

- **12.6 Final Deployment Testing**

After completing the deployment, the complete TaxPal application was tested in the deployed environment. The team verified the major user flows and ensured that the frontend, backend, database, AI chatbot, and email services were working together correctly.

The final deployment successfully made TaxPal available as a cloud-based full-stack application with **Angular frontend, Express.js backend, MongoDB Atlas database, Gemini AI integration, and Resend email services.**

13. CHALLENGES & SOLUTIONS

Challenge	Impact	Resolution
API Field Mismatches	Login/Signup error handling issues and empty transaction rows due to parameter mismatches (e.g., <code>incomeBracket</code> on frontend vs <code>income_bracket</code> on backend).	The Team Lead performed API validation audits and standardized variable naming across the stack.
Asynchronous API Delays	The LLM assistant timed out or returned incomplete responses when analyzing large transaction sets.	Implemented custom database pipelines to aggregate user statistics prior to calling the Gemini API, reducing payload sizes.
Email Deliverability	Reminder notification emails went to spam folders.	Configured SPF and DKIM verification records on the notification domain via Resend, stabilizing mail delivery.
Local MongoDB Setup	Database setup issues during early team branches due to local server version mismatches.	Migrated to shared MongoDB Atlas clusters, standardizing the connection string format for all team members.

14. TEAM MEMBERS & CONTRIBUTIONS

Sachin Shekhar (Team Lead & Frontend Lead)

- Configured overall UI themes and main dashboard.
- Led frontend-backend integration, fixed field mismatches, and resolved routing issues.
- Implemented the AI Chatbot with email support.
- Maintained master repository branch and completed project documentation.

Debjani Roy (Frontend Developer)

- Designed and built forms for registration, login, and income-expenses.
- Implemented the transaction ledger UI table component, including search and category filtering features.
- Built the UI components for tax estimations and report generation module.

Jalal Shaik Jasmine (Frontend Developer)

- Built the UI components for dashboard components and tax calendar.
- Configured Angular services for budgets and reminder list rendering.
- Implemented the report preview panel with print and download options.

Varshitha Gopalam (Backend Developer)

- Established Node.js server layout, middleware, and JWT validation.
- Authored CRUD controllers and route configurations for budgets and transactions.

Rehan Shaik (Backend Developer)

- Integrated Gemini API into backend services, configuring system instructions.
- Implemented PDF generation with `pdfkit`, CSV generators, and email delivery via Resend API.

Jainee Jain (Database Administrator)

- Deployed and configured the MongoDB Atlas cluster.
- Designed and validated schemas and indexes for User, Transaction, Budget, TaxEstimate, Alert, and Report collections.

15. FUTURE ENHANCEMENTS

While TaxPal currently addresses the core requirements of personal finance management and tax estimation, several advanced features can be introduced in future versions to improve automation, convenience, and global usability.

1. **Multi-Currency Conversions:** Support multiple currencies and provide real-time exchange-rate conversion based on the user's selected currency. This would be particularly useful for freelancers working with international clients and receiving payments from different countries.
2. **Optical Character Recognition (OCR):** Integrate AI-based vision and OCR technology to scan physical receipts and invoices. The system could automatically extract information such as merchant name, date, amount, and expense category, reducing manual data entry.
3. **Plaid Integration:** Connect TaxPal with open-banking platforms such as Plaid to securely synchronize bank account transactions. This would allow income and expenses to be imported automatically, reducing the effort required to maintain the financial ledger.
4. **Automated Tax Portal Filing:** In future, TaxPal could integrate with official e-filing platforms such as the **IRS** in the United States or the **Income Tax Department portal** in India. This could enable users to prepare, review, and potentially submit tax documents directly from the application dashboard, subject to the availability of official APIs and applicable regulations.
5. **Advanced Financial Analytics:** The platform could introduce predictive analytics to identify spending trends, forecast future income and expenses, and provide personalized recommendations for improving savings and tax planning.
6. **Mobile Application:** A dedicated Android and iOS application could be developed to allow users to manage transactions, monitor budgets, receive tax reminders, and interact with the AI assistant conveniently from their mobile devices.

16. CONCLUSION

The development of **TaxPal** successfully addresses the major financial challenges faced by freelancers, consultants, and self-employed individuals. By combining **transaction tracking, budget management, quarterly tax estimation, automated reminders, and financial reporting** into a single platform, TaxPal provides users with a clear and organized view of their finances.

The integration of the **Google Gemini API** further enhances the platform by providing an interactive AI assistant that can analyze financial information, offer contextual suggestions, and assist users with supported operations. This makes TaxPal more than just a basic financial ledger.

Through **modular development, secure data handling, API validation, Git collaboration, and systematic testing**, the team has developed a reliable, responsive, and functional full-stack application. Overall, TaxPal provides a strong foundation for simplifying financial management and can be further enhanced with features such as OCR, bank synchronization, multi-currency support, and automated tax filing in the future.

17. REFERENCES

1. Angular Documentation: <https://angular.dev>
2. ExpressJS Routing Manual: <https://expressjs.com>
3. Mongoose Queries Reference: <https://mongoosejs.com>
4. Google AI Studio / Gemini API Guide: <https://ai.google.dev>
5. PDFKit Documentation: <https://pdfkit.org>
6. Resend API Reference: <https://resend.com/docs>
7. IRS Self-Employed Tax Guide (1040-ES): <https://irs.gov>
8. Indian Income Tax Department Slabs: <https://incometax.gov.in>